

INF4030 Conception Orientée Objet (COO)

Concepts Fondamentaux de l'Orienté Objet

Aminata ZERBO/SABANE



Concepts Fondamentaux de l'Orienté Objet

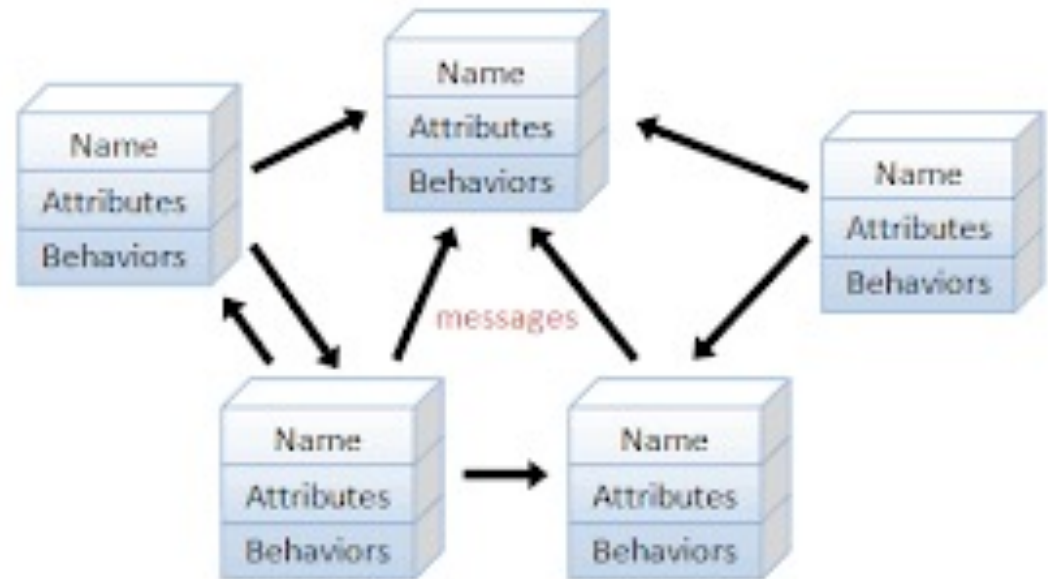


<http://http://etechgroupe.com/fr/blog-2/>

Rappel

Un système orienté objet est un système composé d'objets autonomes en interaction les uns avec les autres par échanges de messages

La fonctionnalité du logiciel résulte de l'interaction entre les différents objets qui le constituent



An object-oriented program consists of many well-encapsulated objects and interacting with each other by sending messages

Les Objets



Définitions

Un **objet** définit une représentation abstraite d'une entité du monde réel (matériel ou immatériel) ou du monde virtuel

Un objet peut être :

- un être vivant, une entité physique
- une idée, un concept, une relation
- un événement, une situation
- une règle, une tâche, un rôle
- ...

Définitions

Exemples d'objets

le boxeur Mohamed Aly

Un client

Un compte bancaire

Une recette

La tour Eiffel,

Banfora

Barack Obama

Un étudiant

Excel

Une commande,

Le Nil

Une équation en Math

Définitions

L'objet représente un élément pertinent du domaine du problème avec des propriétés ou qui doit être manipulé

- Il encapsule une partie de la connaissance du domaine du problème
- Un objet se caractérise par un ensemble de propriétés appelées attributs

Ne pas confondre objet et attributs de l'objet

- Les attributs sont des informations qui caractérisent l'objet

Exemples : Attributs d'une voiture, d'un boxeur, d'une commande ?!

Caractéristiques d'un Objet

Un objet est caractérisé par :

- un état
- un comportement
- une identité

Objet = Etat + Comportement + Identité

L'Etat d'un Objet

L'état d'un objet correspond aux valeurs de tous ses attributs à un instant donné

L'état d'un objet est variable

Les valeurs des attributs d'un objet peuvent changer dans le temps

Cette variabilité est souvent une conséquence des comportements passés

Exemples?

L'Etat d'un Objet

L'état d'un objet correspond aux valeurs de tous ses attributs à un instant donné

L'état d'un objet est variable

Exemples

- Le statut d'une commande
- La couleur d'une voiture
- L'âge d'une personne
- Le poids d'un boxeur

L'Etat d'un Objet

Cependant certains attributs ont des valeurs constants

Exemples

- La marque d'une voiture
- La race d'une personne
- Le lieu de naissance
- La date de naissance

Le Comportement d'un Objet

Le comportement d'un objet définit l'ensemble des opérations qu'il est capable de réaliser

Il décrit les actions et réactions de cet objet

Exemple :

- Rouler, s'arrêter pour une voiture
- Manger, marcher et crier pour un animal
- Deposer ou Retirer pour un compte bancaire

Le Comportement d'un Objet

Les opérations d'un objet sont déclenchées suite une simulation externe (message envoyé par un autre objet)

L'état et le comportement sont liés

Le comportement d'un objet peut dépendre de son état ou modifier son état

Exemples :

- Impossibilité de faire un retrait si le solde est nul ou négatif
- L'opération atterrir d'un avion n'a de sens que si l'avion est en vol (l'état en vol est valide)

L'Identité d'un Objet

Un objet possède une identité qui le caractérise de façon unique

Cette identité permet de le distinguer des autres objets même s'ils ont le même état

L'identité est un concept implicite que chaque objet possède

Elle peut néanmoins être matérialisée à partir d'un identifiant unique issu du domaine du problème appelé **clé naturelle**

Cette clé peut être rajoutée à l'état de l'objet

L'Identité d'un Objet

L'identité peut néanmoins être matérialisée à partir d'un identifiant unique issu du domaine du problème appelé **clé naturelle**

Cette clé peut être rajoutée à l'état de l'objet

Exemples :

- Numéro de série pour un ordinateur
- Numéro d'immatriculation pour une voiture
- Matricule pour un employé ou un étudiant

La Persistance des Objets

Persister un objet c'est lui donner la capacité de vivre au delà d'une exécution

Un objet persistant sauvegarde son état de façon permanente et ce même si le processus qui l' a créé est arrêté (passivation de l'objet)

L'objet peut être alors reconstruit (activation de l'objet) par un autre processus

Les objets non persistants sont dits transitoires ou éphémères

La Persistance des Objets

Par défaut les objets ne sont pas persistants

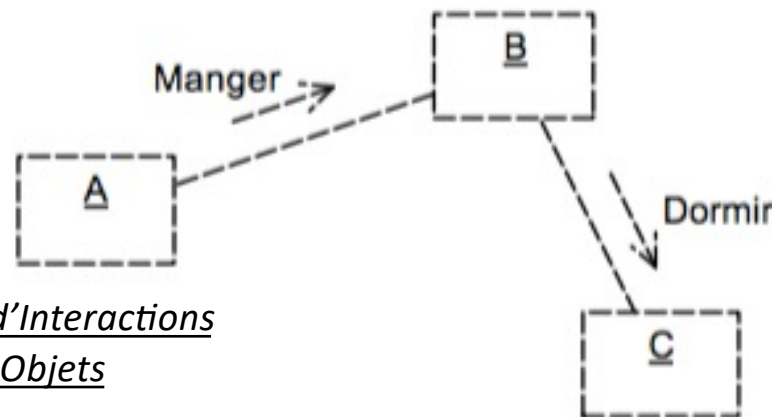
En général, les langages de programmation orientée objet ne fournissent pas de mécanisme de persistance

Autres moyens : Les bases de données offrent des solutions pour la sauvegarde des objets (modèle objet ou hybride)

La Communication entre Objets

Les systèmes orientés objets sont des systèmes basés sur les objets qui travaillent en synergie pour réaliser les fonctionnalités du système

Ils doivent pour cela interagir, communiquer

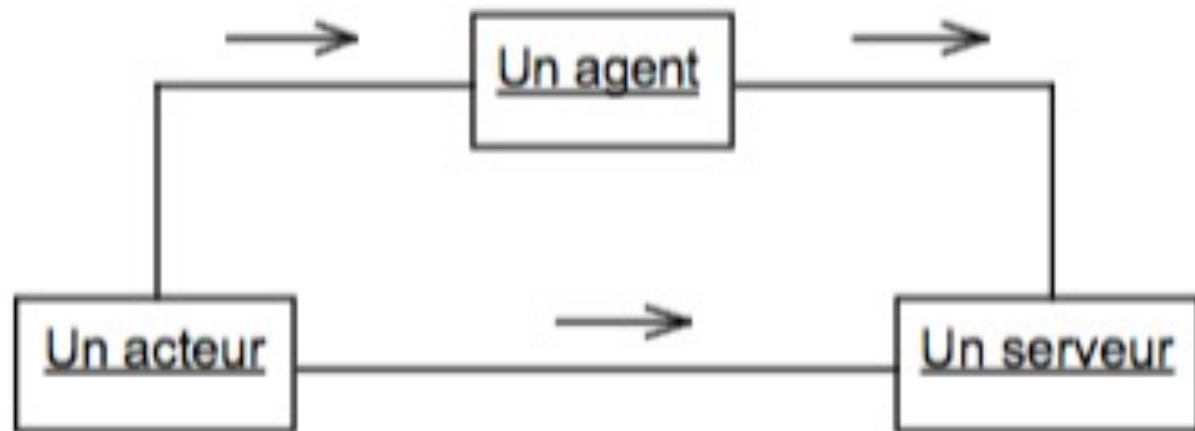


Représentation d'Interactions
entre des Objets

La Communication entre Objets

Les objets peuvent être classés dans trois groupes selon la nature des interactions (direction des messages échangés) :

- Les serveurs
- Les acteurs
- Les agents



Classification des Objets selon les Interactions

La Communication entre Objets

- **Les serveurs** : Objets passifs qui reçoivent les messages des autres. Ce sont des destinataires.
- **les acteurs** : Objets actifs toujours à l'origine des interactions.
- **Les agents**: A la fois acteurs et serveurs. Ils jouent parfois le rôle d'intermédiaire entre le client (acteur) et le fournisseur (serveur). Ils sont à la base du mécanisme de délégation

La Communication entre Objets

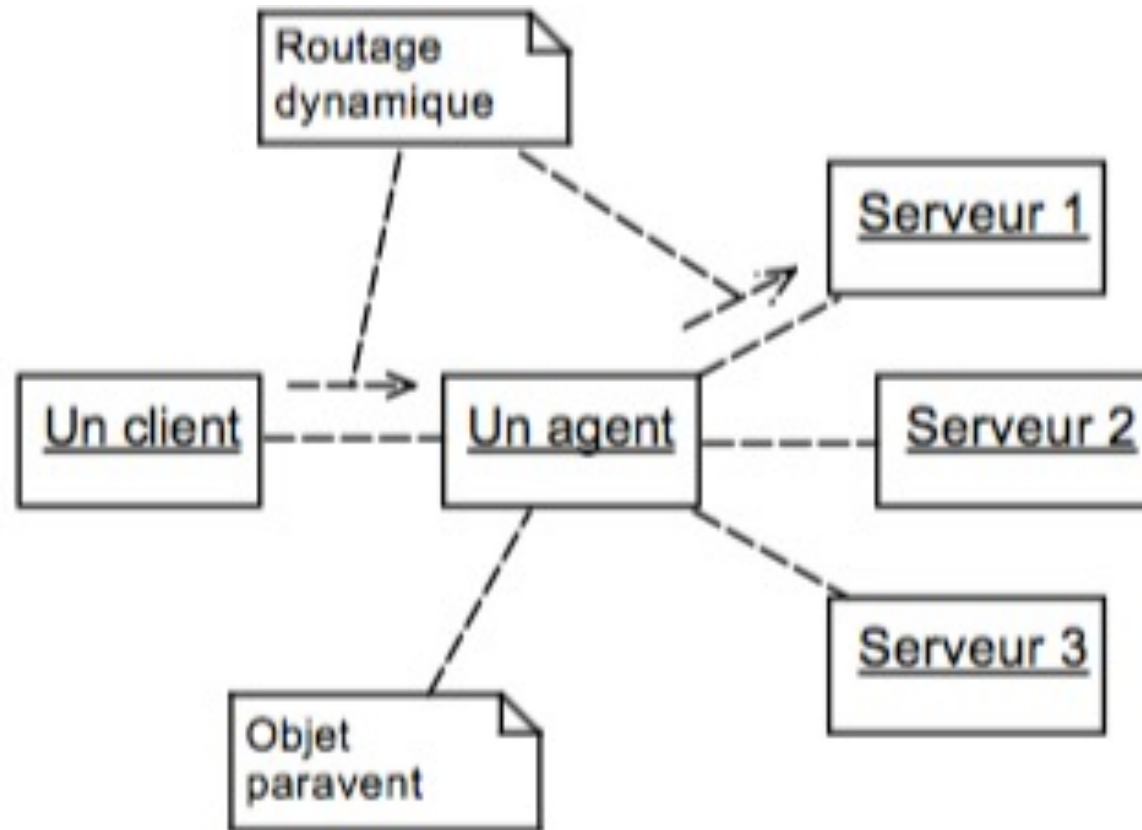


Illustration du
mécanisme de
délégation

Le Concept de Message

La communication entre les objets se fait par messages

Un message est une demande d'action ou de service caractérisée par les paramètres nécessaires à la réalisation de cette action

Un message est un support de communication qui relie les objets de façon dynamique

C'est est une notion abstraite qui peut être mise en œuvre par un appel de procédure, une interruption, un événement,...

Une invocation de message active habituellement une méthode

Le Concept de Message

On distingue cinq catégories principales de messages :

- Les constructeurs : ils permettent de créer des objets
- Les destructeurs : ils détruisent les objets
- Les sélecteurs : ils renvoient tout ou partie de l'état
- Les modificateurs : Ils changent tout ou partie de l'état de l'objet
- Les itérateurs: ils visitent l'état d'un objet ou le contenu d'une structure de données qui contient plusieurs objets

Exercices sur les Objets

Trouver les attributs des propriétés de l'objet voiture dans trois contextes différents :

- Société de vente de véhicules
- Parc automobile d'un ministère
- Société de vente de voitures d'occasion

Analyser la classe **Vector** et préciser pour chacune de ses méthodes la catégorie ou les catégories à laquelle elle appartient

Les Classes



L'Abstraction

En philosophie, l'abstraction désigne à la fois :

- une opération qui consiste à isoler par la pensée une ou plusieurs qualités d'un objet concret pour en former une représentation intellectuelle
- et le produit de cette opération

En informatique, l'abstraction identifie et regroupe des caractéristiques et traitements communs applicables à des entités ou concepts variés

- Cela permet de simplifier et d'unifier la manipulation de telles entités

L'Abstraction

L'abstraction permet de réduire la complexité de la réalité et de ne s'intéresser qu'à l'essentiel selon le contexte et ce afin de mieux la comprendre

L'abstraction est arbitraire

- Elle dépend du point de vue et du contexte
- Un objet peut être vu au travers de différentes abstractions
- Il est donc important d'identifier les caractéristiques pertinentes selon le contexte

Définition de Classe

Une classe est l'abstraction d'un ensemble d'objets qui possèdent une structure identique (liste des attributs) et un même comportement (liste des opérations)

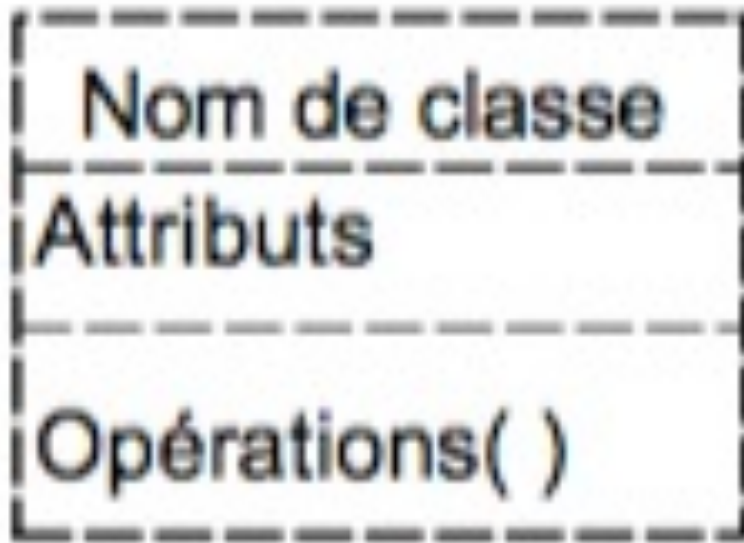
La classe contient les généralités et les objets les particularités

C'est un modèle qui décrit les propriétés et le comportement des objets qu'il représente

La classe peut être perçue comme un moule pour fabriquer des objets

- Ce processus de fabrication ou de construction est appelé **instanciation**
- Un **objet** est une **instance** d'une et une seule classe

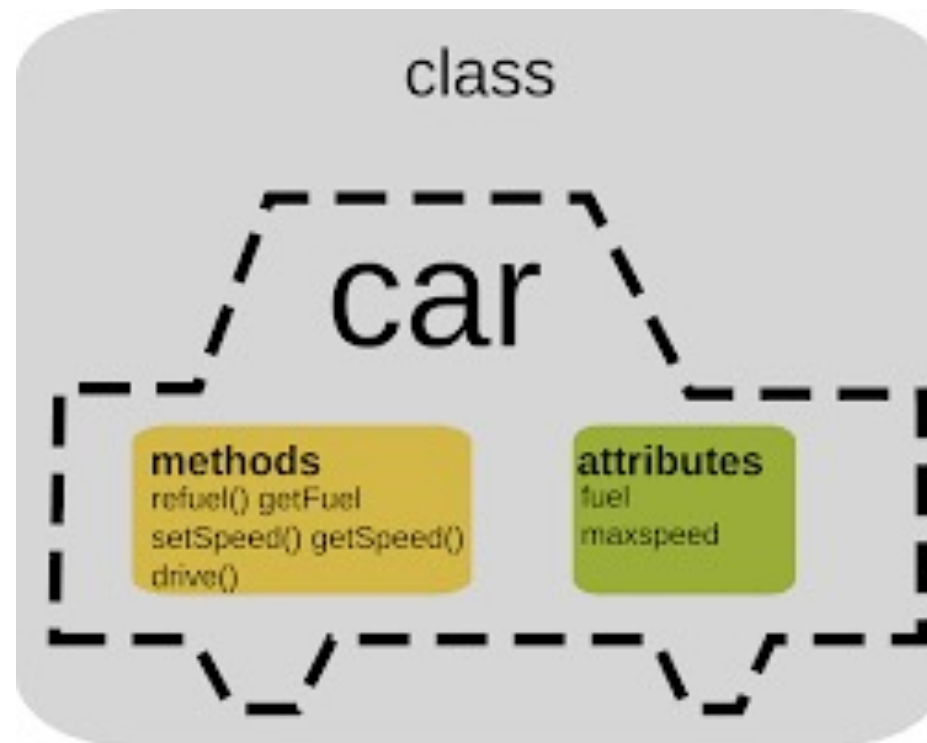
Définition de Classe



*Représentation Graphique
des Classes*

Classe
=
Description des attributs des
objets
+
Opérations (méthodes)
applicables à l'objet

Définition de Classe



<https://gist.github.com/tigarcia/f1163d0e45bd4e7fdb45591d35e3fe5a>

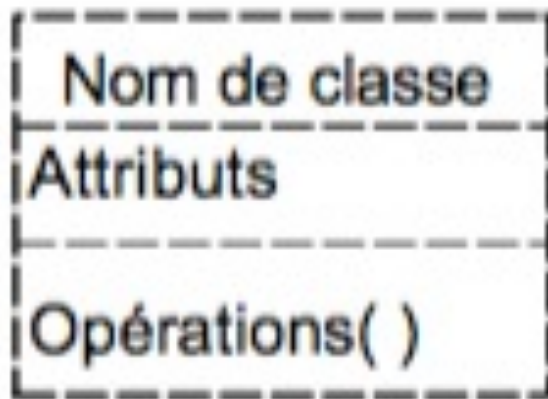
Exercices sur les Classes

Définir les attributs et les opérations des classes suivantes :

- Motocyclette
- Téléviseur
- Fichier
- NombreComplexe
- ListeEntiers

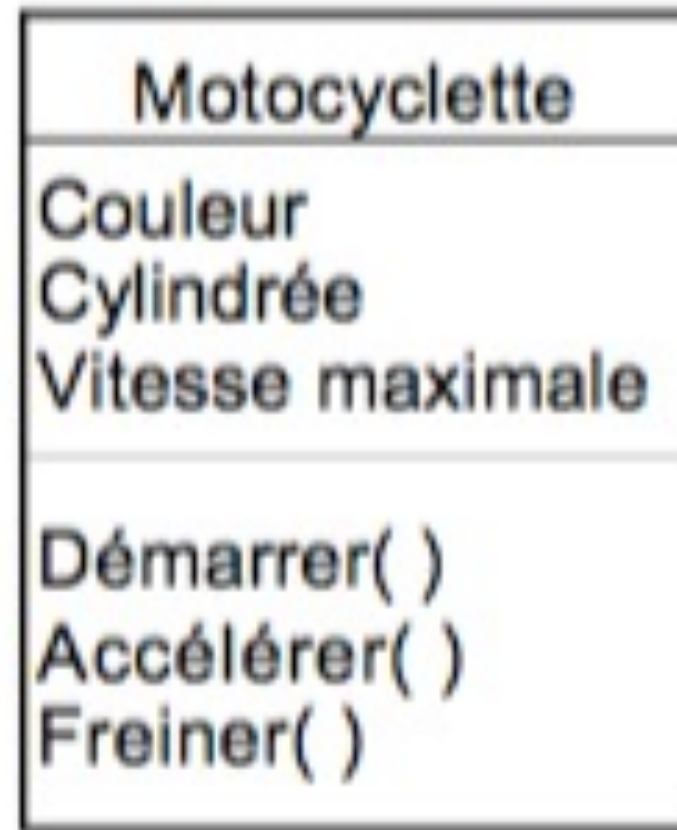
Donner deux instances de chacune de ces classes

Définition de Classe



*Représentation Graphique
des Classes*

*Classe **Motocyclette***



Définition de Classe

Téléviseur
Allumer()
Eteindre()
Changer de programme()
Régler le volume()

Nombre complexe
Additionner()
Soustraire()
Multiplier()
Diviser()
Prendre le module()
Prendre l'argument()
Prendre la partie réelle()
Prendre la partie imaginaire()

**Interfaces des classes Téléviseur et
Nombre Complexe**

Définition de Classe

Une classe est décrite à l'aide de deux parties :

- **Sa spécification** : elle décrit le domaine de définition et les propriétés des instances de cette classe. Elle correspond à la notion de type
- **Sa réalisation** : Elle décrit comment la spécification est réalisée. Elle contient le corps des opérations ainsi que les données nécessaires à leur fonctionnement

La classe offre une interface des objets de la classe constituée des méthodes et attributs non-privés

Attributs d'une Classe

Les attributs sont des informations qui décrivent les objets que représentent une classe

Un attribut est caractérisé par un **nom**, un **type** et une **visibilité**
Le **type** d'un attribut détermine les valeurs que peut prendre l'attribut et les opérations et les opérations qui lui sont applicables

La **visibilité** d'un attribut détermine les règles d'accès à cet attribut

Exemple : Privé Marque : chaîne de caractères
Privé Datefabrication : Date

Types d'Attributs d'une Classe

Attribut d'instance

C'est un attribut propre à chaque objet
Sa valeur varie donc d'un objet à un autre

Exemples : les attributs x, y et nom de la classe Point

Attribut statique ou de classe

C'est un attribut commun à tous les objets de la classe
Il n'existe qu'un exemplaire de cet attribut pour tous les objets de cette classe

Méthodes d'une Classe

Une méthode ou opération décrit un comportement des objets de la classe

Exemples : Rouler, Accélérer pour la classe Voiture

Manger, Marcher, Crier pour la classe Animal

LireAbscisse, ModifierOrdonnée pour la classe Point

Une méthode est caractérisée par un nom, une visibilité, un type retour et des paramètres

Exemples : public réel LireAbscisse()

public void ModifierOrdonnée(réel y)

Types de Méthodes d'une Classe

Méthode d'instance

Une méthode d'instance est une méthode dont l'appel donne des résultats différents d'un objet à un autre

Méthode de classe

Aussi appelée méthode statique, une méthode de classe est commune à tous les objets de cette classe.

- Le résultat de l'appel d'une telle méthode reste le même d'un objet à un autre
- La méthode n'est pas attaché à un objet et peut être appelée via la classe
- Elle ne peut faire référence qu'à des méthodes ou attributs statiques.

L'Encapsulation



L'Encapsulation

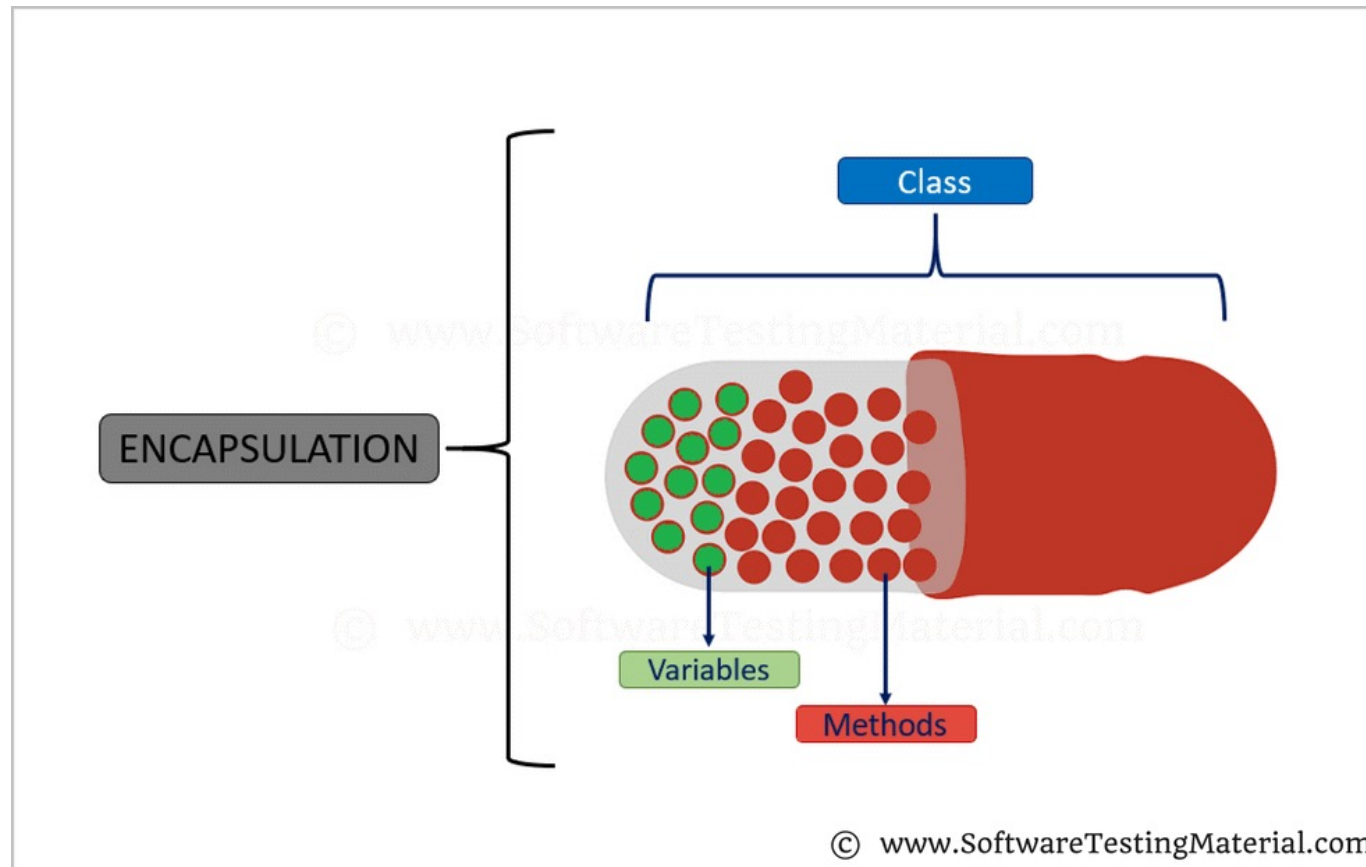
C'est un mécanisme qui séparer l'implémentation (réalisation) de la spécification

- Il permet de rassembler les données et les méthodes au sein d'une structure en cachant l'implémentation
- Rendre les données uniquement accessibles au travers des opérations définies dans la classe

Ne doit être visible de l'extérieur que ce qui est nécessaire

- Les détails d'implémentation sont « cachés »
- L'ensemble des opérations d'une classe rendu visible aux autres classes porte le nom d'interface.

L'Encapsulation



Avantages de l'Encapsulation

L'encapsulation permet de :

- Garantir l'intégrité des données encapsulées dans les objets en les protégeant des accès non autorisés
- Masquer l'implémentation
- Réduire le couplage
- Renforcer l'autonomie et l'indépendance de chaque classe
- Favoriser la modularité et faciliter la maintenance

Les utilisateurs d'une classe ne dépendent pas de sa réalisation mais de sa spécification

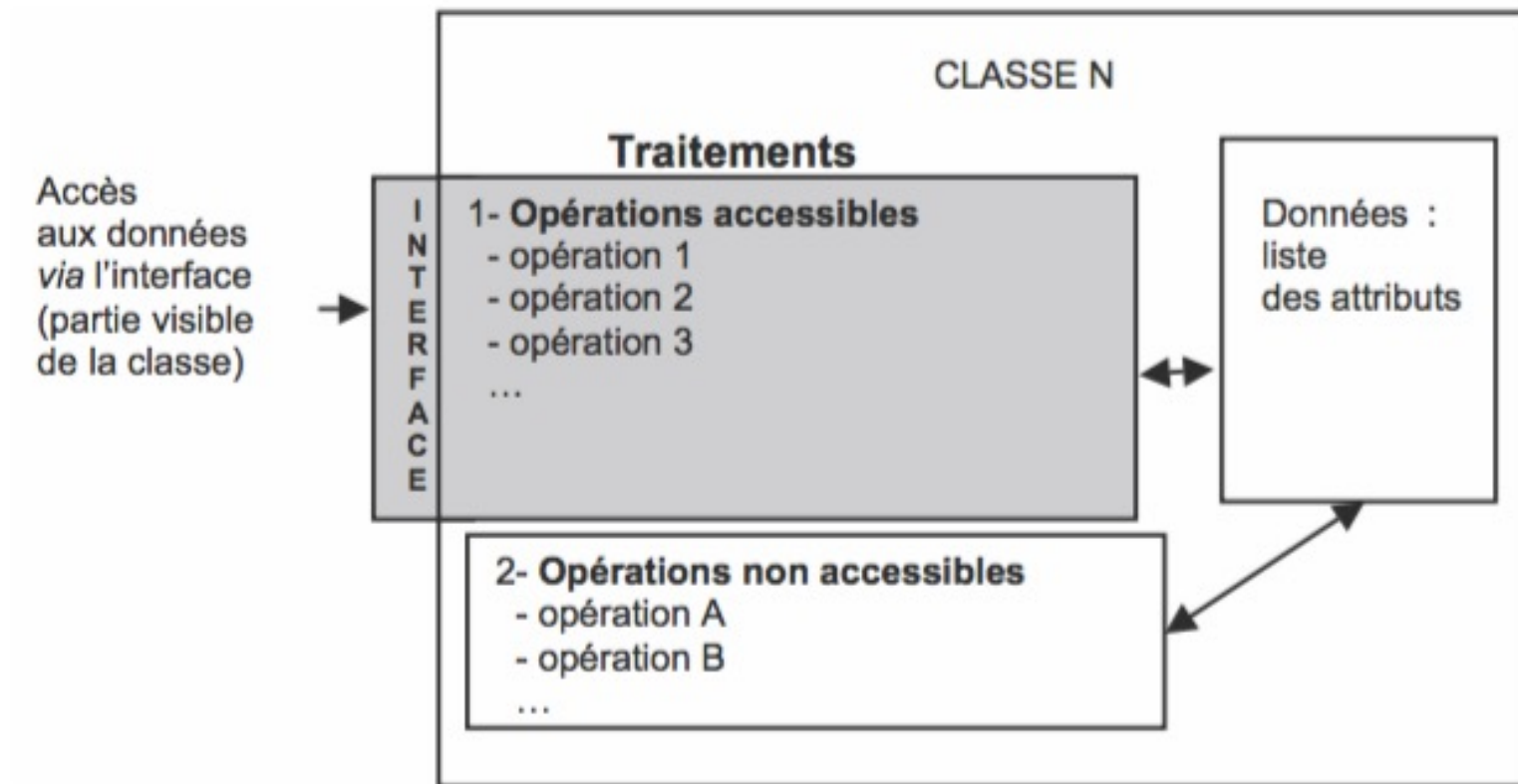
L'Encapsulation

Par défaut, les attributs sont encapsulés et ne peuvent pas être directement manipulés par les autres classes

Les autres classes doivent passer par les opérations disponibles via l'interface de la classe pour accéder aux attributs (accesseurs ou sélecteurs, modificateurs ou mutateurs)

L'encapsulation est bénéfique lorsqu'il y a une forte cohésion dans les classes et un faible couplage entre les classes

L'Encapsulation



L'Encapsulation : Règles de Visibilité

Les règles de visibilité précisent le degré d'encapsulation des éléments d'une classe

- Briser l'encapsulation peut permettre de réduire le temps d'accès aux attributs

Selon les langages, différents niveaux d'encapsulation existent.

Les trois principaux niveaux sont:

- Le niveau privé
- Le niveau protégé
- Le niveau public

Règles de visibilité
+ Attribut public # Attribut protégé - Attribut privé
+ Opération publique() # Opération protégée() - Opération privée()

L'Encapsulation : Règles de Visibilité

Les trois principaux niveaux d'encapsulation sont:

Le niveau privé : Niveau le plus élevé. Il indique que l'attribut ou l'opération ne peut être accessible qu'à l'intérieur de la classe

Le niveau protégé : les attributs ou opérations de ce niveau sont accessibles à l'intérieur de la classe et aussi par les classes dérivées de la classe

Le niveau public: Niveau le plus faible. Les attributs ou opérations de ce niveau sont accessibles par toutes les autres classe.

Les Relations entre les Classes



Les Relations entre les Classes

Les relations entre les classes sont une abstraction des liens qui existent entre les objets

Elles définissent la manière dont les objets des classes interagissent les uns avec les autres

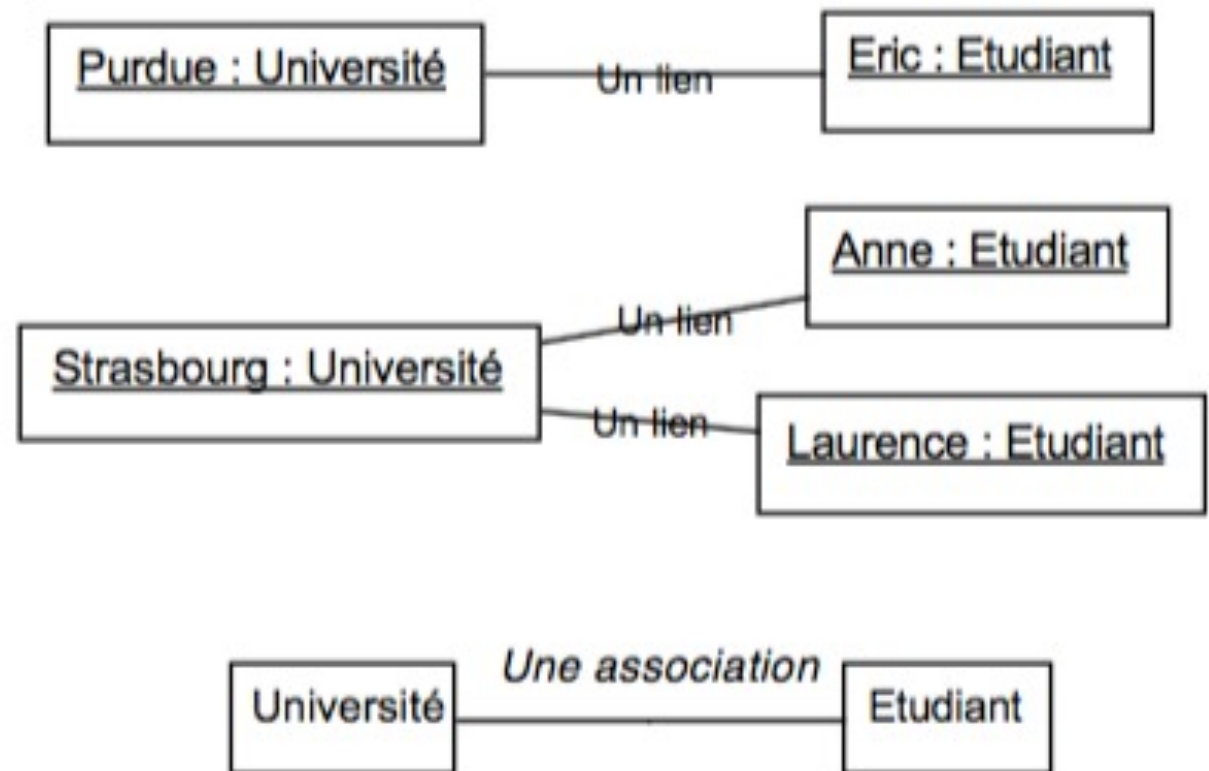
On distingue deux relations principales

- L'association
- L'agrégation

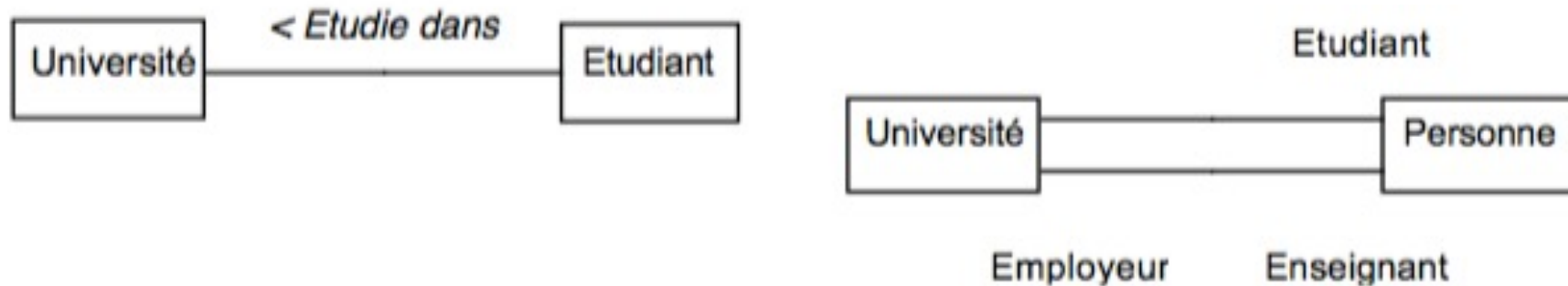
L'Association

L'association représente une relation entre deux ou plusieurs classes

Elle exprime une connexion sémantique bidirectionnelle entre classes



L'Association



Pour clarifier la nature de l'association :

- On peut la décorer avec une expression verbale avec un sens de lecture
- On peut préciser le rôle d'une classe dans une association
Important lorsque plusieurs associations relient deux classes

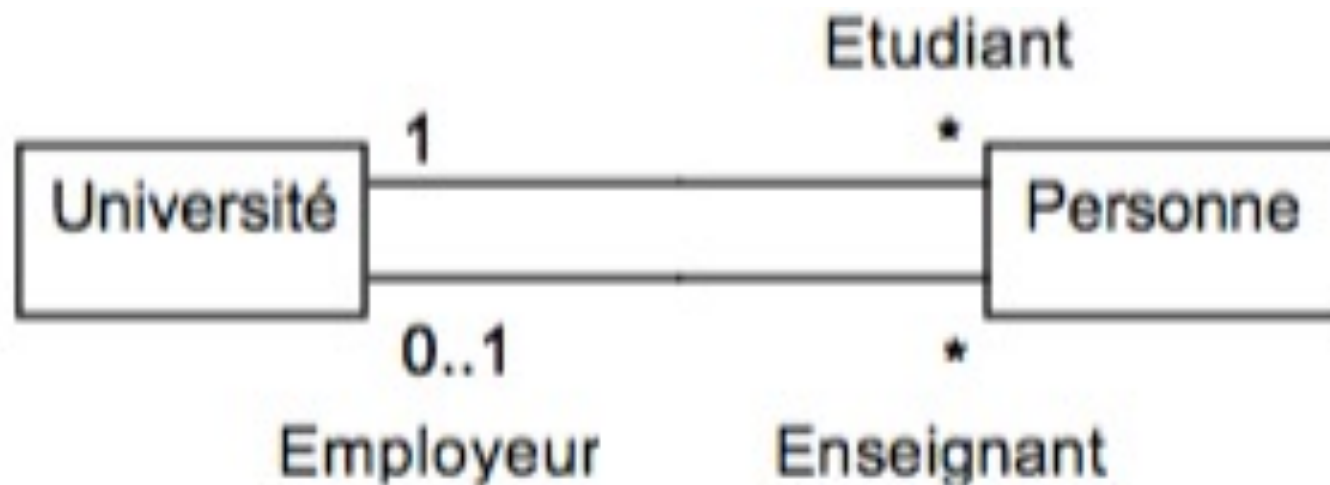
L'Association

Les multiplicités (ou cardinalités) précisent le nombre de participants qui prennent part à la relation

Multiplicité	Signification
1	Un et un seul
0..1	Zéro ou un
M..N	De M à N (entiers naturels°)
*	De zéro à plusieurs
0..*	De zéro à plusieurs
1..*	De un à plusieurs

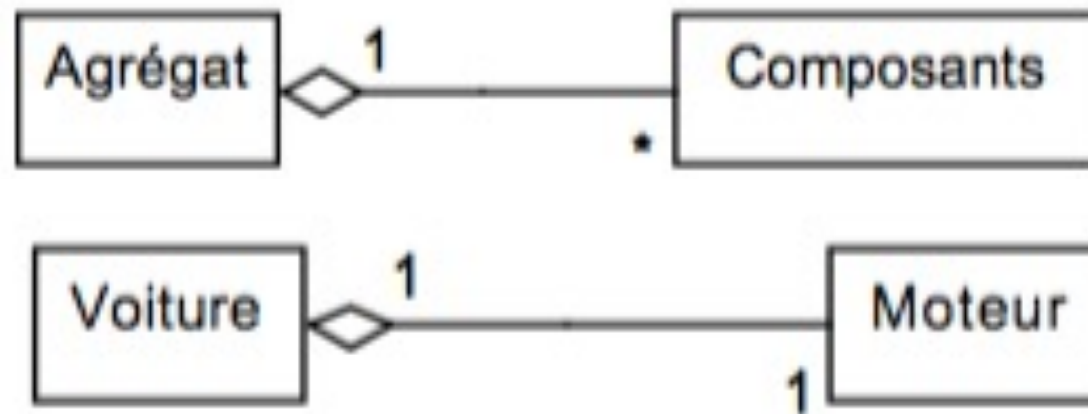
L'Association

Les multiplicités (ou cardinalités) précisent le nombre de participants qui prennent part à la relation



L'Agrégation

L'agrégation est une relation qui permet d'exprimer des relations maître-esclave, tout et parties, composé et composants



L'agrégation permet d'assembler des objets de base pour construire des objets plus complexes

L'Agrégation

Mathématiquement, l'agrégation est une relation transitive, non symétrique et réflexive

Exemples :

- Une entreprise et ses directions
- Une direction et ses services
- Un pays et ses villes

La Composition

La composition est une forme particulière d'agrégation dans laquelle le composant ne peut appartenir qu'à un seul composé

- La composition est plus forte que l'agrégation
- On suppose généralement que les composants sont créés et meurent avec le composé (la destruction du tout se propage en cascade à ses parties)

Exemples :

Un pays et ses villes

Un livre et ses pages

Les Relations entre les Classes

Les relations expriment un couplage entre les classes

Le degré de ce couplage dépend de la nature de la relation

- L'association correspond à un couplage faible
- L'agrégation est une forme d'association qui indique un couplage plus fort
- La composition indique un couplage encore plus fort que celui de l'agrégation

La Hiérarchisation des Classes



Hiérarchisation des Classes

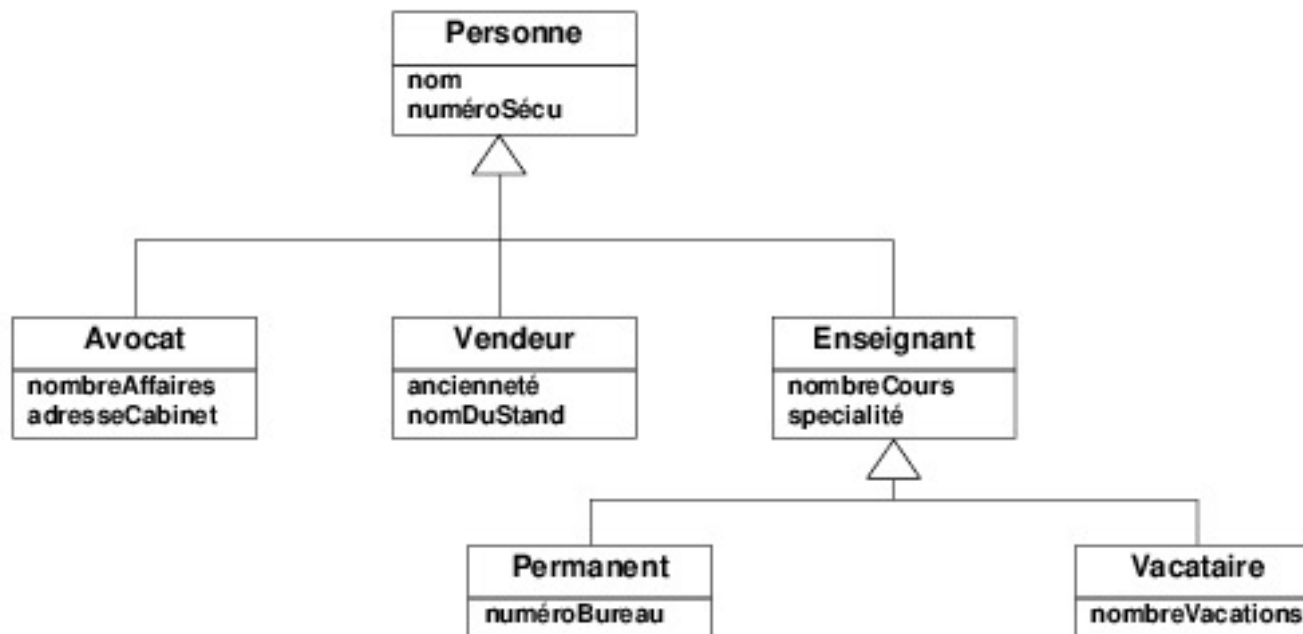
La hiérarchisation ou classification de classes permet de gérer la complexité en ordonnant les objets au sein d'arborescence de classes d'abstraction croissante (du plus général au plus spécifique)

Hiérarchiser c'est ranger ou ordonnancer les abstractions

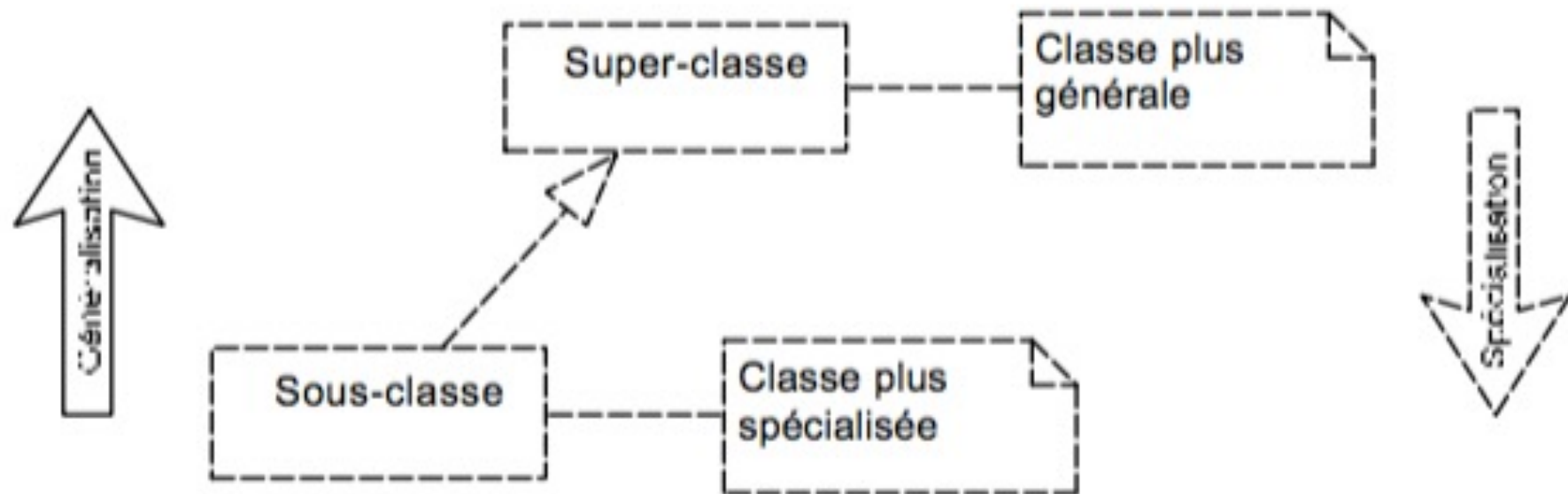
La hiérarchisation est généralement mise en œuvre par l'héritage

La relation d'héritage décrit une inclusion d'ensembles

Hiérarchisation des Classes



Généralisation & Spécialisation



Deux mécanismes inverses l'un de l'autre permettent d'hiérarchiser les classes : la **généralisation** et la **spécialisation**

Généralisation des Classes

La généralisation de classes consiste à factoriser dans une classe, appelée **super-classe**, les éléments communs (attributs et opérations) d'un ensemble de classes

La généralisation signifie toujours **est un** ou **est une** sorte de
Elle ne concerne que les classes

Pas d'instanciation en liens et donc pas de multiplicité

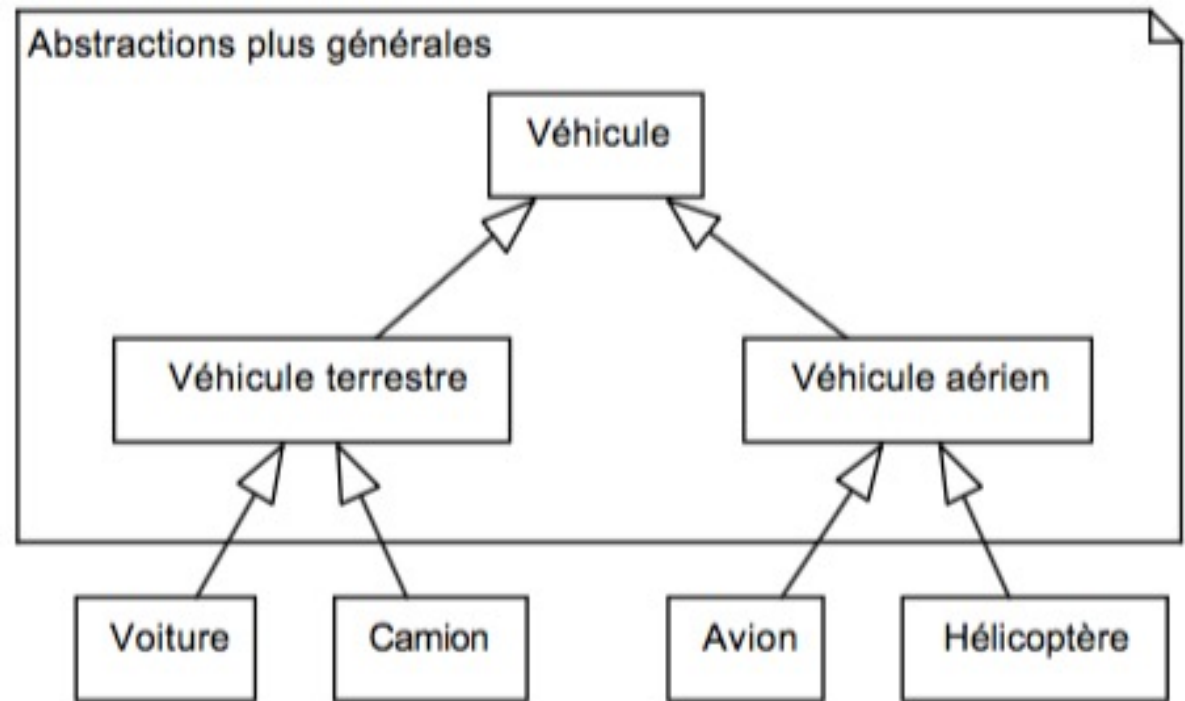
Exemple : Le lion est une sorte de carnivore

Classes Abstraites

Les niveaux supérieurs dans la hiérarchie peuvent être purement abstraits

Ces niveaux sont représentés par les classes abstraites

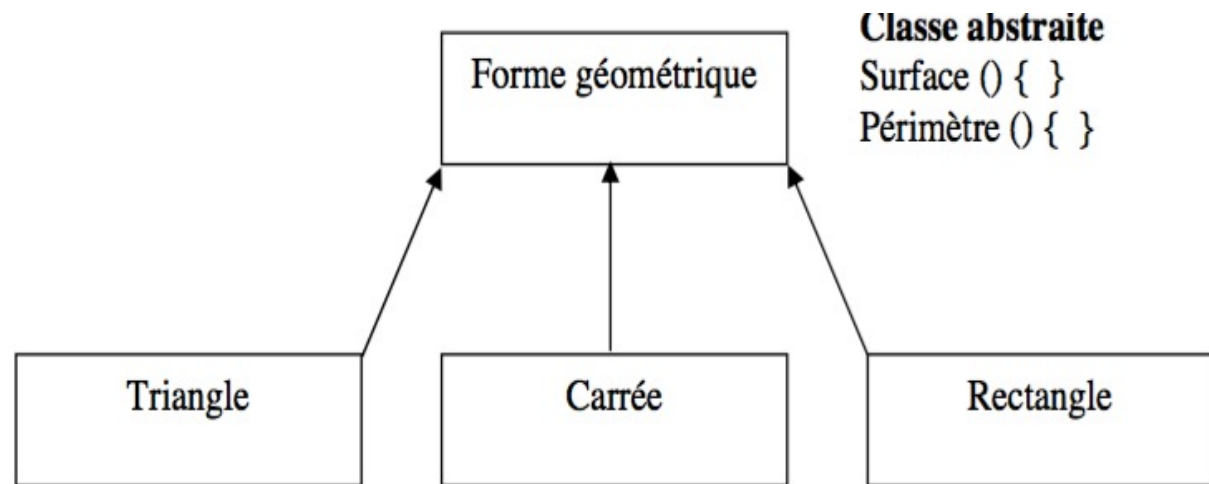
Ils permettent d'ordonner et comprendre



Classes Abstraites

C'est une classe qui ne peut être instanciée

- Elle contient au moins une méthode abstraite



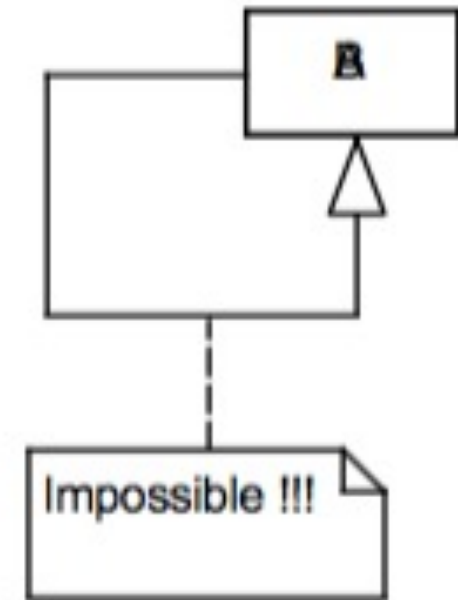
Une méthode abstraite est une méthode sans implémentation

- Elle doit être forcément redéfinie dans les sous classes concrètes

Caractéristiques de la Généralisation

La généralisation est une relation non réflexive

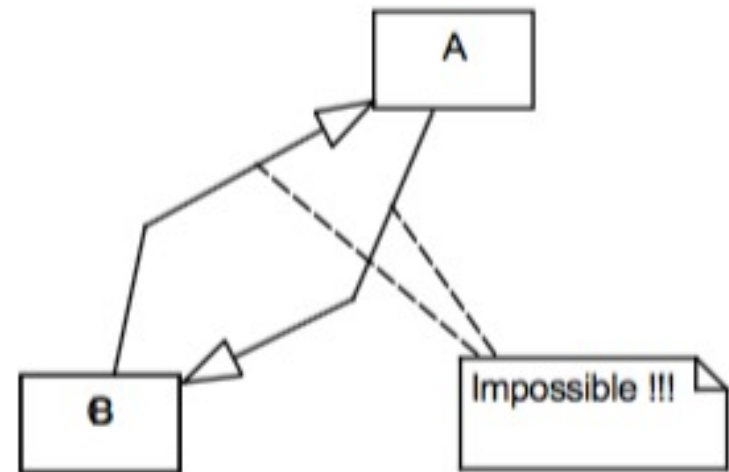
- Une classe ne peut pas dériver d'elle même



Caractéristiques de la Généralisation

La généralisation est une relation non symétrique

Si A dérive d'une classe B, alors B ne peut pas dériver de A



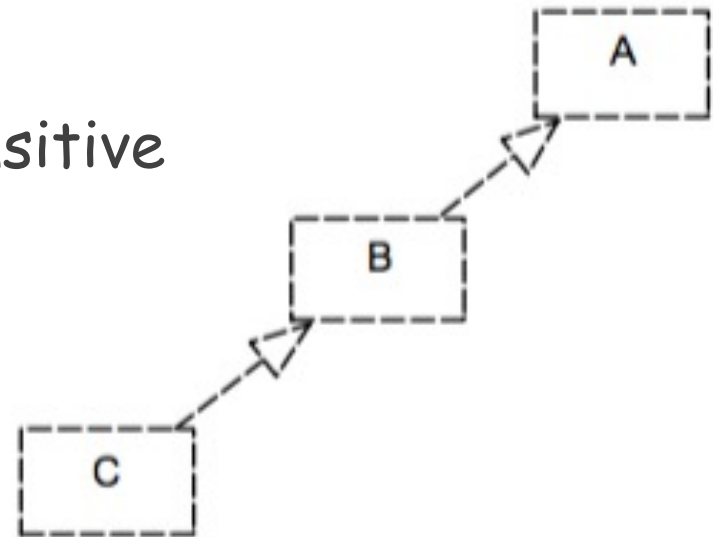
Caractéristiques de la Généralisation

La généralisation est une relation transitive

Si une classe B dérive d'une classe A

et une classe C dérive B

alors C dérive de A



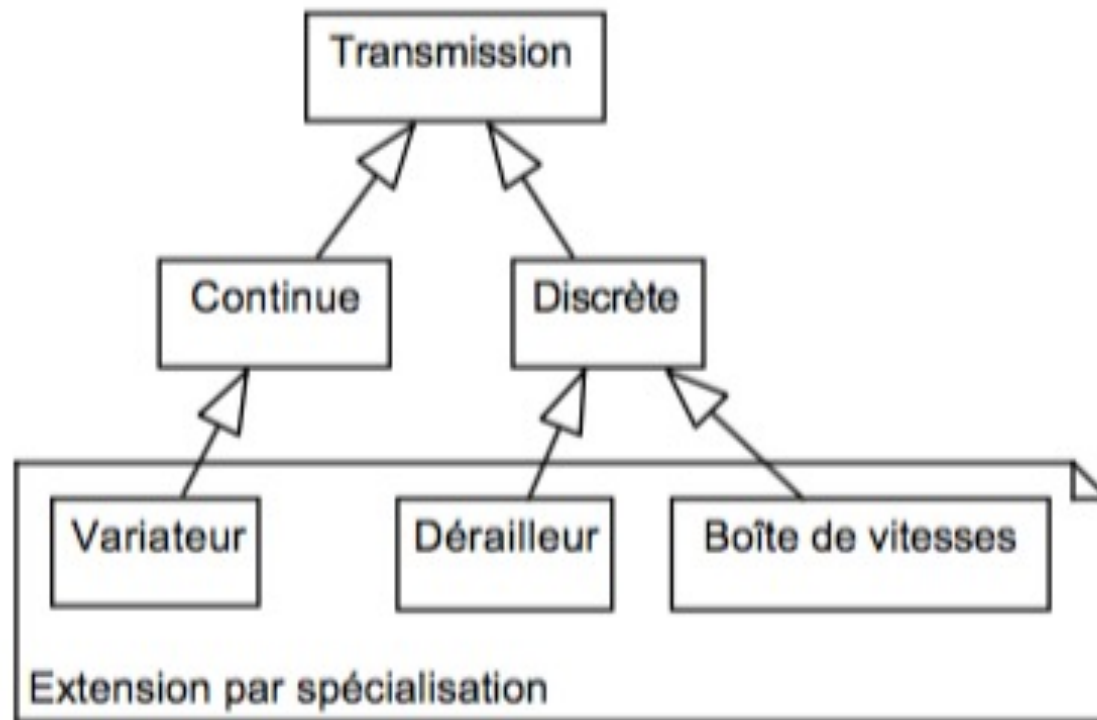
Spécialisation des Classes

La **spécialisation** représente la démarche inverse de la généralisation

Elle consiste à créer à partir d'une classe, plusieurs classes spécialisées avec de nouvelles caractéristiques

Les classes dérivées héritent des attributs et des opérations de la classe mère

Spécialisation des Classes



Spécialisation des Classes

Chaque nouvelle classe créée est dite spécialisée

Elle comporte en plus des attributs ou opérations de la super-classe (disponibles par héritage), des attributs ou opérations qui lui sont propres

Elle peut également redéfinir certaines méthodes de sa super-classe (surcharge de méthodes)

Une classe spécialisée porte aussi le nom de sous-classe

La généralisation-spécialisation (mis en œuvre par l'héritage) est un des mécanismes les plus importants de l'approche objet qui facilite la réutilisation des classes

Difficultés de la Hiérarchisation

Les bonnes Hiérarchisations sont stables et extensibles

La Hiérarchisation se fait selon un point de vue

Les animaux peuvent être classés selon la station, le type de nourriture, la protection

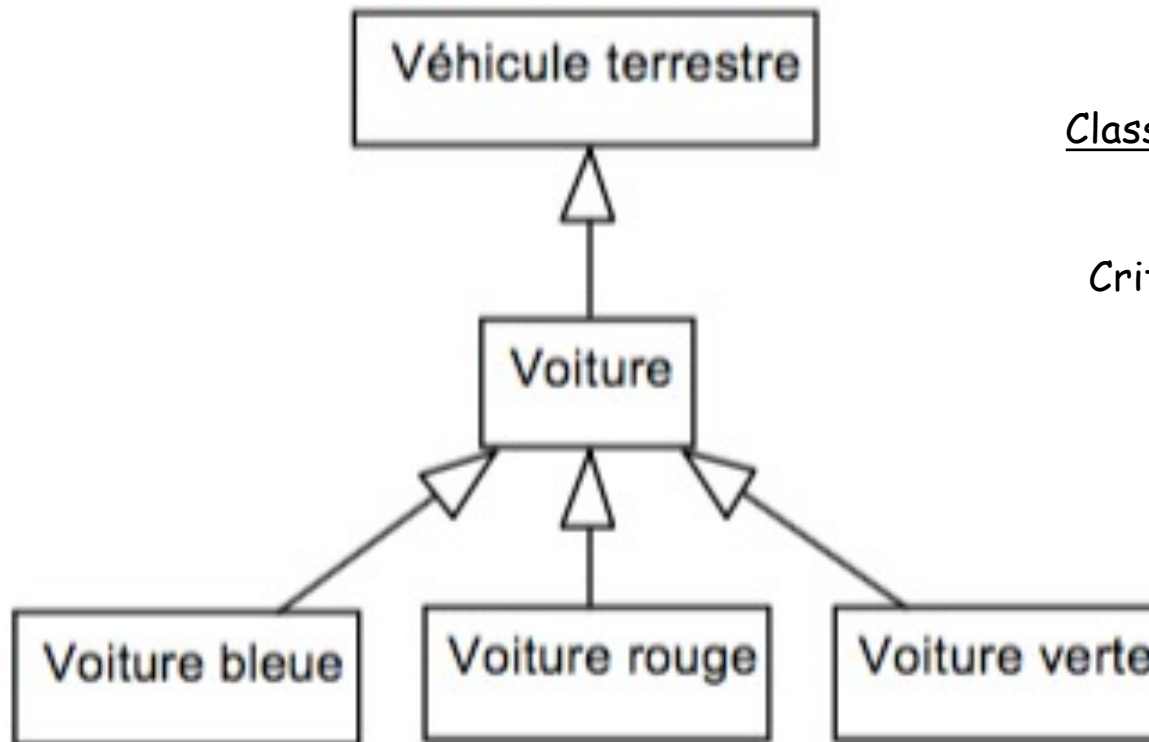
Difficultés de la Hiérarchisation

Déterminer des critères pertinents et l'ordre d'application n'est pas toujours trivial

Les choix peuvent conduire à des situation de covariance qui complexifie la maintenance

La généralisation multiple peut être une solution mais doit également être utilisée avec discernement

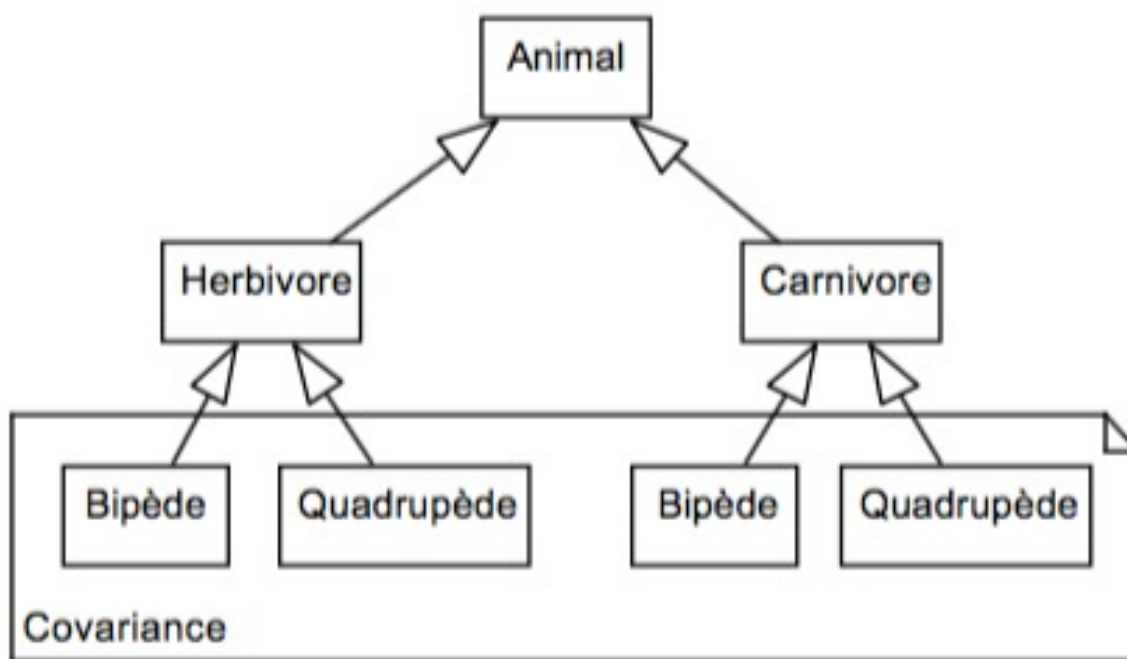
Difficultés de la Hiérarchisation



Classification des Voitures
selon la Couleur

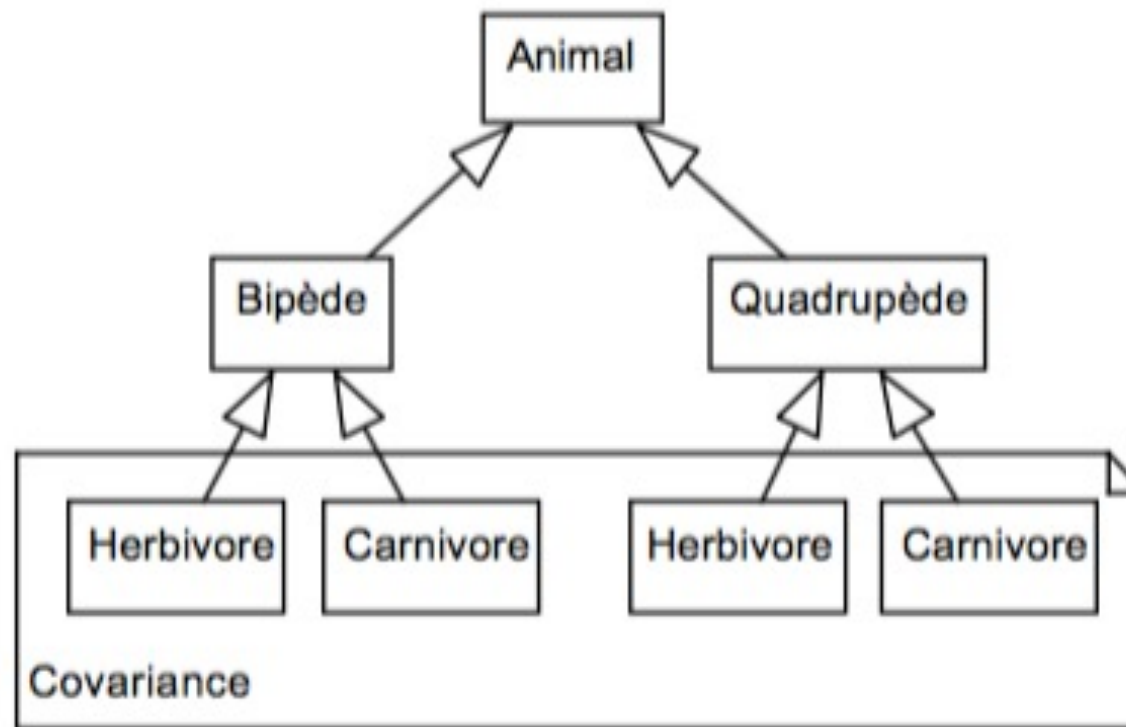
Critère de classification
peu pertinent

Difficultés de la Hiérarchisation



Classification des Animaux
selon le Type de Nourriture et
le type de station

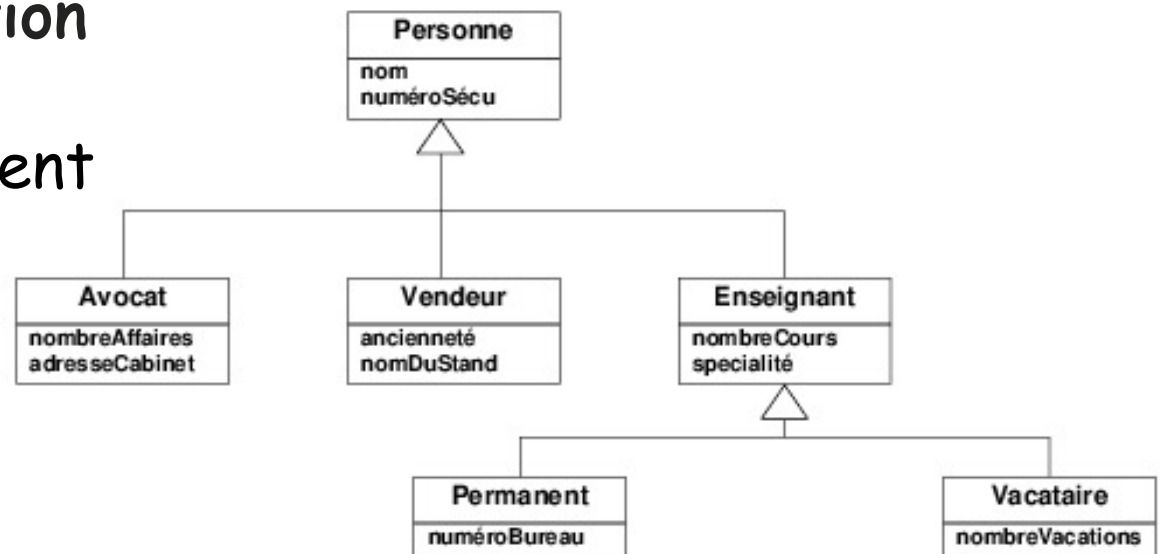
Difficultés de la Hiérarchisation



Classification des Animaux
selon le Type de Nourriture et
le type de station

Types d'Hiérarchisation

Généralisation/Spécialisation simple
les sous-classes ne possèdent qu'une super-classe



Types d'Hiérarchisation

Généralisation / Spécialisation Multiple

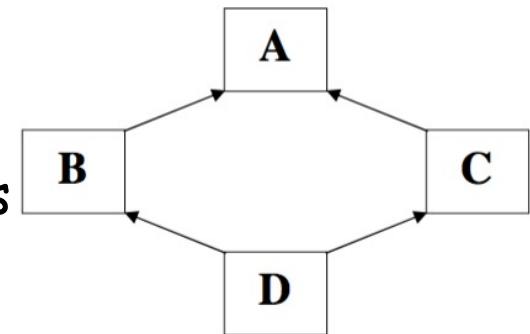
Les sous-classes possèdent plus d'une super-classe

Ce mécanisme permet de regrouper au sein d'une seule et même classe les attributs et méthodes de plusieurs classes

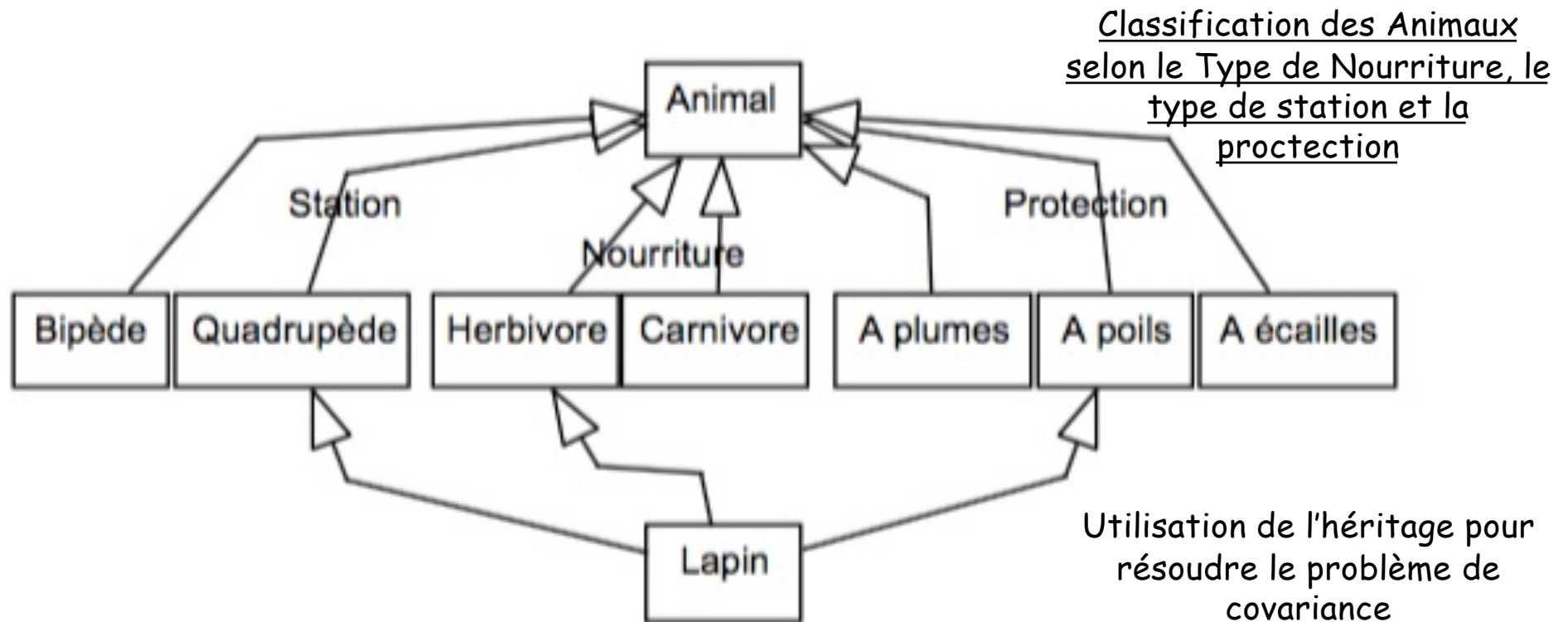
Il n'existe pas dans tous les langages de programmation orientés objet

Un mécanisme problématique s'il n'est pas bien maîtrisé

- *Collision de nommage* : conflit entre deux attributs de même nom hérité de parents différents
- *Héritage à répétition*



Exemple d'Hiérarchisation Multiple



Héritage vs Composition

La classification propage l'état, le comportement et les contraintes de la classe mère aux sous classes

Il n'y a pas d'héritage partiel; Toutes les propriétés et tous les comportements de la classe parente sont hérités intégralement dans la classe fille

→ L'héritage concerne l'état, le comportement et les contraintes

L'héritage n'est pas la composition

Le besoin d'hériter partiellement est le signe que la relation considérée n'est pas vraiment une relation d'héritage

Le Principe de Substitution

Enoncé à l'origine par Barabara Liskov, le principe de substitution permet de déterminer qu'une relation d'héritage correspond bien à une généralisation

Si une propriété P est vraie pour une instance x d'un type T , alors cette propriété P doit rester vraie pour tout instance y d'un sous-type de T

où dans notre cas les sous-types de T sont les classes qui dérivent de T

Le Principe de Substitution

Si S est un sous-type de T , alors tout objet de type T peut être remplacé par un objet de type S sans altérer les propriétés désirables du programme concerné

Tout invariant inhérent à la classe T doit être respecté dans ses classes filles.

Ainsi, tout contrat imposé par une classe C ou une méthode M doit être vérifié dans toute classe héritant de C

Le respect de ce principe permet de bénéficier pleinement du polymorphisme qui est l'un des mécanismes les plus puissants de l'orienté objet

Exemple de Violation du Principe de Substitution

Rectangle est une classe représentant les propriétés d'un rectangle

Attributs : hauteur, largeur

Postcondition : la hauteur et la largeur sont modifiables indépendamment via leurs modificateurs

Carré est une sous classe de Rectangle et représente les propriétés d'un carré

Postcondition : les quatre côtés du carré doivent être égaux

→ on ne peut modifier la hauteur et la largeur indépendamment →
contrédit la postcondition du rectangle

Un autre non sens c'est d'avoir un carré avec les attributs hauteur et largeur

Le Polymorphisme



Le Polymorphisme

Le mot Polymorphisme vient du grec et signifie qui peut prendre plusieurs formes

Exemple : ?

Le Polymorphisme est l'un des mécanismes les plus puissants de l'orienté objet

Ce mécanisme permet de déclencher des opérations différentes en réponse à un même message

L'opération est alors dite polymorphe

Le Polymorphisme

Le polymorphisme se base sur le mécanisme d'héritage

Chaque sous-classe peut redéfinir les méthodes héritées de sa super-classe afin de mieux prendre en compte certaines particularités liées à ses objets

Une méthode ainsi redéfinie aura un comportement différent si elle est appelée sur un objet de la classe mère ou sur des objets des sous classes l'ayant redéfinie

Exemples : ?

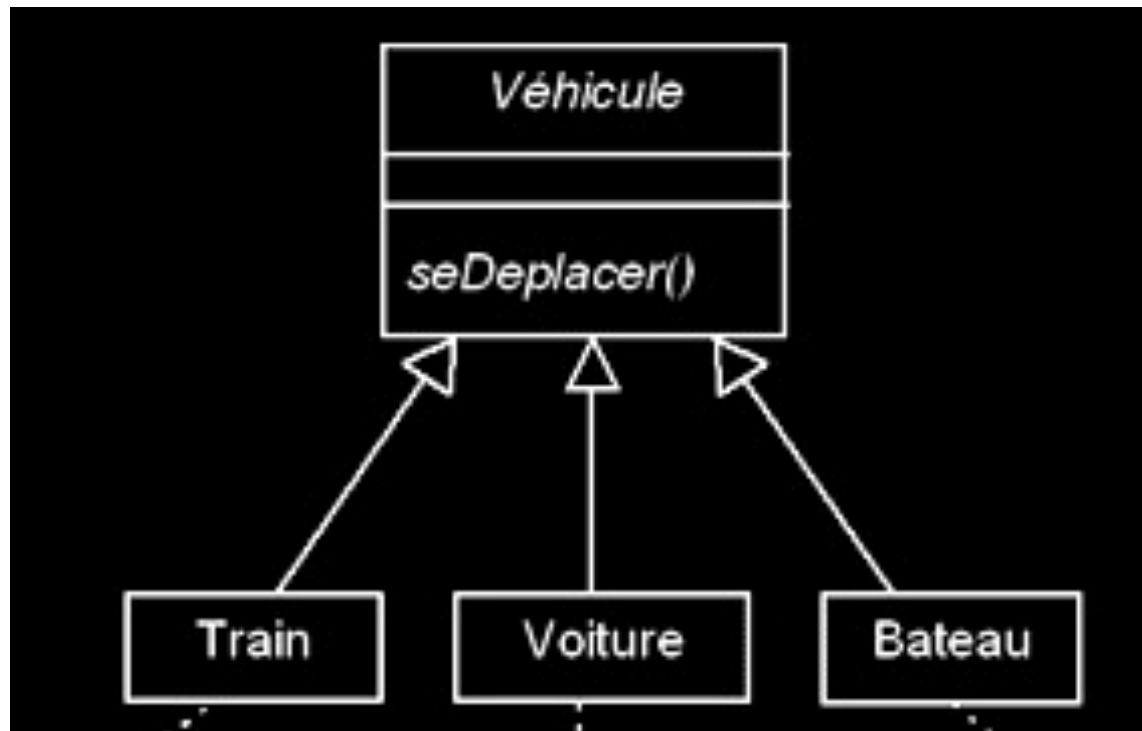
Le Polymorphisme

Avantages du polymorphisme

- Suppression de points de maintenance multiples
- Réduction de l'usage des instructions à branchement multiple

Le Polymorphisme

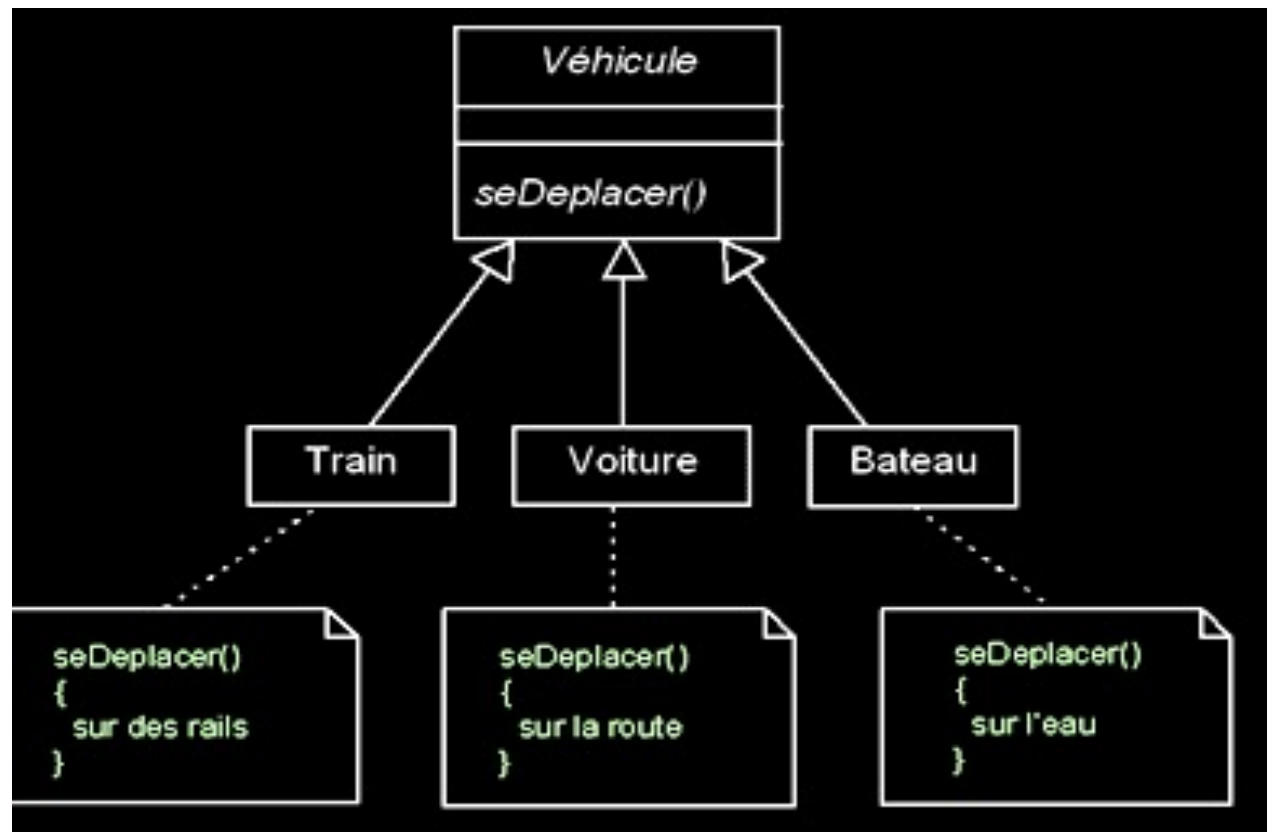
Exemple :
Collection Polymorphe



Le Polymorphisme

Exemple :

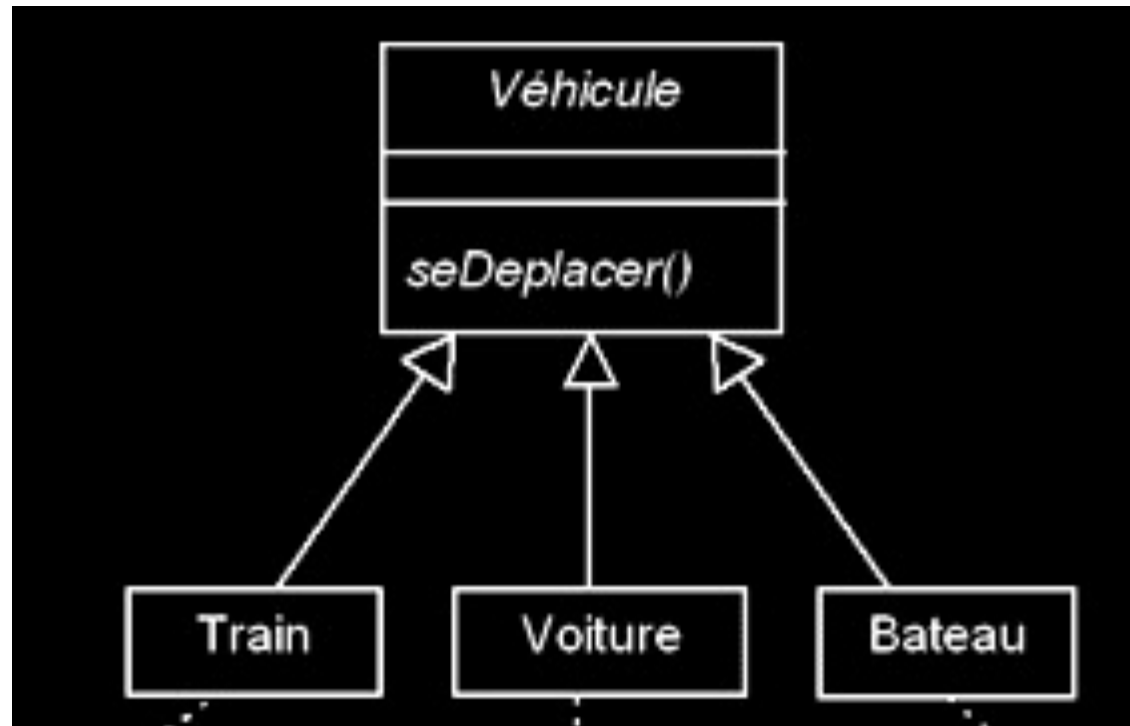
La méthode `seDeplacer()` est une méthode polymorphe



Le Polymorphisme

Exemple :

Ajout d'une nouvelle
sous-classe Avion



Exercice sur Le Polymorphisme

Voir Fiche TD n°1

Références

Cf. Présentation du Cours